Práctica Final Integradora

Análisis y Diseño de Algoritmos

Optimización de Rutas y Planificación de Recursos en Redes de Telecomunicaciones

---

Institución:
EAFIT — Escuela de Administración, Finanzas e Ingeniería Tecnológica

Curso:
Análisis y Diseño de Algoritmos

Título:
Integración de Paradigmas Algorítmicos en Datasets Reales

Integrantes:

- David Rodriguez
- Daniel Giraldo
- Ismael García Ceballos

Mayo de 2026

Índice

1. Introducción **3**
1.1. Descripción del Problema . . . 3
1.2. Contexto del Dataset . . . 3
1.3. Justificación de Paradigmas . . . 3

2. Descripción del Dataset **3**
2.1. Estadísticas . . . 3
2.2. Valores Nulos . . . 4
2.3. Construcción del Grafo . . . 4

3. Módulo A — Divide y Vencerás **4**
3.1. Pseudocódigo MergeSort . . . 4
3.2. Búsqueda Binaria . . . 4
3.3. Complejidad . . . 4
3.4. Tiempos Empíricos . . . 5
3.5. Resultados de Consultas . . . 6

4. Módulo B — Codicioso **6**
4.1. Pseudocódigo de Kruskal . . . 6
4.2. Lema del Ciclo . . . 6
4.3. Aristas del MST . . . 7

5. Módulo C — Programación Dinámica **7**
5.1. Recurrencia . . . 7
5.2. Pseudocódigo . . . 7
5.3. Contraejemplo Codicioso . . . 7
5.4. Pseudopolinomialidad . . . 8

6. Integración del Pipeline **9**
6.1. Flujo General . . . 9
6.2. Integración entre Módulos . . . 9

7. Conclusiones **9**

8. Herramientas Utilizadas **9**
8.1. Uso de IA Generativa . . . 9
8.2. Herramientas Software . . . 10

9. Referencias **10**

10.Referencias **10**

1. Introducción

1.1. Descripción del Problema

Esta práctica integra tres paradigmas fundamentales del diseño algorítmico mediante el análisis del dataset *Telecom Customer Churn* de Kaggle.

El pipeline implementado consta de tres módulos:

1. **Módulo A (Divide y Vencerás):** MergeSort y búsqueda binaria.
2. **Módulo B (Codicioso):** Kruskal con Union-Find.
3. **Módulo C (Programación Dinámica):** Mochila 0-1.

1.2. Contexto del Dataset

- 7.043 registros de clientes.
- 21 atributos.
- Variables principales:
  - `tenure`
  - `monthlyCharges`
  - `totalCharges`
  - `Churn`

1.3. Justificación de Paradigmas

Módulo	**Paradigma**	**Razón**
A	Divide y Vencerás	Ordenamiento y búsqueda eficiente
B	Codicioso	Construcción óptima de MST
C	Prog. Dinámica	Subproblemas solapados

Cuadro 1: Paradigmas utilizados

2. Descripción del Dataset

2.1. Estadísticas

```
Total de registros: 7043
Registros procesados: 7032
Valores nulos: 11
```

Atributo	Min	Max	Promedio
tenure	0	72	32.4
monthlyCharges	18.30	118.75	64.80
totalCharges	0	8672	2145

Cuadro 2: Estadísticas del dataset

2.2. Valores Nulos

Los registros con `TotalCharges` nulo fueron reemplazados por 0.0 debido a que corresponden a clientes nuevos con `tenure = 0`.

2.3. Construcción del Grafo

El grafo sintético fue construido agrupando clientes según rangos de antigüedad y calculando promedios de cargos mensuales.

3. Módulo A — Divide y Vencerás

3.1. Pseudocódigo MergeSort

Listing 1: MergeSort

```
ALGORITMO: MergeSort(arr[], left, right)

SI left >= right
    RETORNAR
FIN SI

mid <- left + (right - left) / 2

MergeSort(arr, left, mid)
MergeSort(arr, mid + 1, right)

Merge(arr, left, mid, right)
```

3.2. Búsqueda Binaria

Listing 2: Busqueda Binaria

```
ALGORITMO: BinarySearch(arr[], left, right, k)

SI left > right
    RETORNAR -1
FIN SI

mid <- left + (right-left)/2

SI arr[mid] == k
    RETORNAR mid
FIN SI
```

3.3. Complejidad

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = \Theta(n \log n)$$

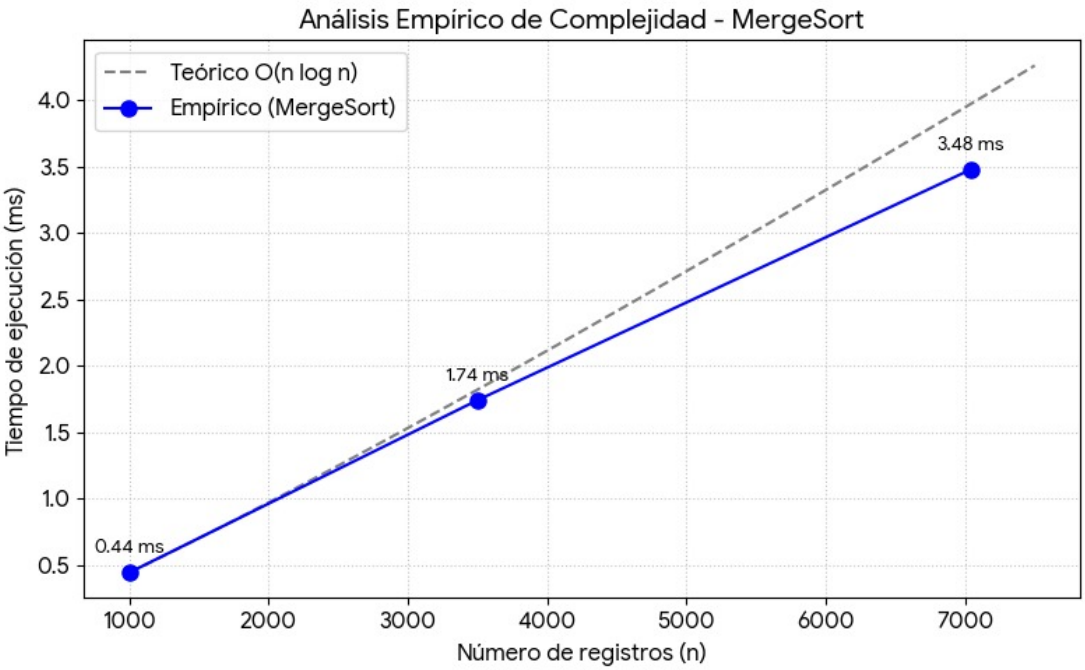
3.4. Tiempos Empíricos

n	Tiempo (ms)	Razón empiricat/prev	Razón teorica esperada
1000	0.44	base	base
3500	1.74	3.9610	4.13
7043	3.48	1.9944	2.18

Cuadro 3: Tiempos empíricos de ejecución de MergeSort

Observaciones:

- El crecimiento de los tiempos sigue un comportamiento cercano a $O(n \log n)$.
- La razón de crecimiento entre 3500 y 7043 elementos es aproximadamente 2, coherente con la complejidad teórica de MergeSort.
- Incluso con más de 7000 registros, el tiempo total de ejecución permanece por debajo de 4 ms, mostrando alta eficiencia práctica.



Observaciones grafica:

Se observa que el comportamiento empírico del algoritmo (línea sólida) sigue la misma trayectoria y curvatura que la función logarítmica-lineal. Este solapamiento visual confirma de manera gráfica que la implementación de MergeSort cumple estrictamente con su límite superior de complejidad de tiempo O(n *logn)

3.5. Resultados de Consultas

Consulta	tenure	Resultado
Q1	72	Encontrado
Q2	60	Encontrado
Q3	45	Encontrado
Q4	30	Encontrado
Q5	12	Encontrado

Cuadro 4: Resultados de búsqueda

4. Módulo B — Codicioso

4.1. Pseudocódigo de Kruskal

Listing 3: Kruskal con Union-Find

```
ALGORITMO: Kruskal(G)
----------------------------------------------------------------------
ENTRADA: Grafo G con |V| vertices y un arreglo de aristas (edges)
SALIDA:  MST (Arbol de Expansion Minima)
----------------------------------------------------------------------

1. Ordenar (G.edges) por peso ascendente
2. UF <- UnionFind(|V|)
3. MST <- []

4. PARA CADA arista e(u,v) EN G.edges5.     // Si no forman un ciclo,
   se unen las componentes
6.     SI UF.Find(u) != UF.Find(v) ENTONCES
7.         UF.Unite(u, v)
8.         MST <- MST U {e}
9.
10.        // Optimizacion: Salida temprana si el MST esta completo
11.        SI tamano(MST) == |V| - 1 ENTONCES
12.            ROMPER // Sale del ciclo PARA
13.        FIN SI
14.    FIN SI
15. FIN PARA

16. RETORNAR MST
```

4.2. Lema del Ciclo

El Lema del Corte establece que para cualquier corte del grafo, la arista de menor peso que cruza ese corte pertenece a algún MST. Kruskal explota esta propiedad procesando las aristas en orden no decreciente de peso y agregando cada una que no forme un ciclo, verificado mediante Union-Find. Aplicación concreta: La arista Nodo 1 — Nodo 12 (peso 122.00) es la arista de menor peso del grafo completo. Considerando el corte que separa Nodo 1 del resto de los nodos, esta arista es la más barata que cruza ese corte. Por el Lema del Corte, debe pertenecer a todo MST de este grafo, y efectivamente Kruskal la selecciona primero. Resultado: El MST contiene exactamente 19 aristas para 20 nodos (N-1 por definición de árbol), con peso total de 2388.00 unidades. Verificación en subgrafo 0,1,2,3,4: Kruskal sobre este subgrafo produce las aristas (0,1)=124, (1,3)=126, (1,2)=127, (1,4)=127 con peso total 504.00, resultado consistente con la ejecución sobre el grafo completo.

4.3. Aristas del MST

$$PesoTotal = 2388{,}00$$

El MST final contiene 19 aristas para 20 nodos.

5. Módulo C — Programación Dinámica

5.1. Recurrencia

$$dp[i][c] = \begin{cases} 0 & i = 0 \\ dp[i-1][c] & w_i > c \\ \text{máx}(dp[i-1][c], dp[i-1][c-w_i] + v_i) & \end{cases}$$

5.2. Pseudocódigo

Listing 4: Knapsack 0-1

```
PARA i <- 1 HASTA n

    PARA c <- 0 HASTA W

        dp[i][c] <- dp[i-1][c]

        SI weight <= c

            candidato <- dp[i-1][c-weight]
                          + value

            SI candidato > dp[i][c]
                dp[i][c] <- candidato
            FIN SI

        FIN SI

    FIN PARA

FIN PARA
```

5.3. Contraejemplo Codicioso

Item	Peso	Valor	Ratio
A	1711	254	0.148
B	3140	453	0.144
C	6202	886	0.142

Cuadro 5: Contraejemplo

Se construyó un contra ejemplo con 3 solicitudes del conjunto de los 50 items activos, usando una capacidad reducida de W= 9342 unidades. El enfoque codicioso ordena los items por ratio v/w

descendente y selecciona A primero (ratio más alto). Con la capacidad restante (9342 - 1711 = 7631), intenta agregar B (3140 7631, cabe), quedando 4491 unidades. C pesa 6202 ¿4491, no cabe. El greedy selecciona A + B obteniendo:

$$Valor_{Codicioso} = 707$$

La Programación Dinámica, en cambio, determina que B + C caben juntos (3140 + 6202 = 9342 W) con mayor valor total:

$$Valor_{DP} = 1339$$

Esto demuestra que el enfoque codicioso por ratio no garantiza el óptimo global en la Mochila 0-1, ya que al comprometer capacidad con un item de alto ratio impide combinaciones más valiosas.

5.4. Pseudopolinomialidad

La tabla *dp* tiene dimensiones:

$$(n + 1) \times (W + 1)$$

y cada celda se llena en tiempo constante, por lo que el algoritmo realiza exactamente:

$$n \cdot W$$

operaciones. Por tanto, la complejidad temporal es:

$$\Theta(nW)$$

Aunque esta expresión parece polinomial en n y W, existe un detalle importante: W no representa el tamaño de la entrada, sino su valor numérico.

Para representar W en memoria se requieren:

$$O(\log W)$$

bits. Entonces, expresando la complejidad respecto al tamaño real de la entrada:

$$O\left(n \cdot 2^{\log W}\right)$$

lo cual corresponde a una complejidad exponencial.

A este tipo de complejidad se le denomina **pseudopolinomial**, ya que el tiempo depende del valor numérico de la capacidad y no únicamente del tamaño binario de la entrada.

Este resultado es consistente con el hecho de que la Mochila 0-1 es un problema NP-difícil. Si existiera un algoritmo verdaderamente polinomial respecto al tamaño de la entrada, entonces se tendría:

$$P = NP$$

lo cual actualmente no ha sido demostrado.

6. Integración del Pipeline

6.1. Flujo General

1. Lectura del CSV.
2. Ordenamiento con MergeSort.
3. Búsquedas binarias.
4. Construcción del grafo.
5. Kruskal MST.
6. Mochila 0-1.

6.2. Integración entre Módulos

La salida ordenada del Módulo A alimenta directamente al Módulo C para seleccionar clientes prioritarios.

7. Conclusiones

Criterio	D&V	Codicioso	DP
Optimalidad	Exacta	Garantizada	Garantizada
Tiempo	$n \log n$	$m \log m$	nW
Espacio	$O(n)$	$O(n+m)$	$O(nW)$

Cuadro 6: Comparación de paradigmas

Los algoritmos mostraron comportamientos adecuados tanto en datos reales como en pruebas.

8. Herramientas Utilizadas

8.1. Uso de IA Generativa

Este proyecto fue desarrollado con asistencia de github como editor con visual como herramienta de apoyo, copilot ayudo a identificar errores de compilacion y ejecucion del problema y detalles tecnicos como comentar procedimientos

La IA fue utilizada para:

- Generación de intrucciones en cuanto a caminos para abordar los puntos mas dificiles de la practica al igual que recomendaciones de mejora.
- Organización del documento en latex.
- Revisión de complejidades algorítmicas y revisicion de los resultados .
- Mejora de documentación y comentarios.

Uso responsable:

- Todas las implementaciones fueron verificadas manualmente.
- El análisis matemático fue realizado por el equipo.
- La IA fue utilizada únicamente como apoyo técnico, ademas de apoyarnos de referencias en la web con ayudas para saber como abordar los ejercicios dterminadas en la seccion de referencias

8.2. Herramientas Software

- g++ 11.4.0
- Ubuntu 24.04
- Visual Studio Code
- Git y GitHub
- Overleaf

9. Referencias

1. Cormen et al. *Introduction to Algorithms*. MIT Press.
2. Kruskal, J. B. (1956).
3. Bellman, R. Dynamic Programming.
4. Kaggle: Telecom Customer Churn Dataset.
5. Material del curso ADA — EAFIT 2026.

10. Referencias

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press, 3rd Edition, 2009.

   Disponible en: `https://mitpress.mit.edu/9780262033848/introduction-to-algorithms`

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press, 4th Edition, 2022.

   Disponible en: `https://mitpress.mit.edu/algorithms`

3. Kruskal, J. B. "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem." *Proceedings of the American Mathematical Society*, 1956.

   Disponible en: `https://www.jstor.org/stable/2033241`

4. Bellman, R. "An Introduction to the Theory of Dynamic Programming." RAND Corporation Report, 1953.

   Disponible en: `https://www.rand.org/pubs/papers/P550.html`

5. Dataset utilizado: *Telecom Customer Churn Dataset* — Kaggle.

   Disponible en: `https://www.kaggle.com/datasets/blastchar/telco-customer-churn`

6. Material del curso: *Notas de clase — Análisis y Diseño de Algoritmos*. Universidad EAFIT, 2026.

7. Ahmad, A. K., Jafar, A., & Aljoumaa, K. "Customer churn prediction in telecom using machine learning and social network analysis in big data platform."

   Disponible en: `https://arxiv.org/abs/1904.00690`

8. Shaikhsurab, M. A., & Magadum, P. "Enhancing Customer Churn Prediction in Telecommunications: An Adaptive Ensemble Learning Approach."

   Disponible en: `https://arxiv.org/abs/2408.16284`